

CRIMSON ORBIT – MUSICAL BAND & REAL SOUND STUDIO

Vishwakarma Institute of Technology, Pune • Group - 9

Guide: Prof. Gopal Upadhye | Primary Platform: emu8086 | Extension: HTML/CSS/JS Studio | Rule: Each Member Speaks Once

Speaker & Roll No.	Assigned Role & Contributions	Time
1. Akash Kumar (Roll No. 9)	Project Leader, Research & Workflow, emu8086 Piano Module & Real Steinway Piano in Web Studio	~2.0 min
2. Bhargude Sanskar (Roll No. 38)	Acoustic Guitar Module in emu8086 (6 Strings & Chords) + Martin Guitar Fretboard in Web Studio	~1.5 min
3. Aher Krishna (Roll No. 7)	Drums Module in emu8086 (Noise Generator & 5 Drum Sounds) + 3D Ludwig Drum Kit in Web Studio	~1.5 min
4. Hari Birare (Roll No. 49)	Songs Repertoire Architecture (songs.asm), Note Timings & Connecting Songs to Web Studio	~1.0 min
5. Agaldare Ghansham (Roll No. 4)	Jukebox Song Player in emu8086, Visualizer Bars, [ESC] Key + Moving Sound Equalizer in Web Studio	~1.5 min
6. Akash Kumar (Closing & Demo)	Project Assembling, Explaining Why We Made Both emu8086 & Web Studio, and Live Demonstration	~1.5 min

SPEAKER 1: AKASH

ASSIGNED ROLE: PROJECT LEADER — IDEA, RESEARCH, WORKFLOW & CONCERT PIANO

WHAT TO SAY (SPEAKS EXACTLY ONCE):

"Good morning/afternoon, Respected Professor and classmates. Our course project is **Crimson Orbit – Musical Band**, developed and tested using the **emu8086 emulator**.

How We Started & Our Research Workflow:

When starting our microprocessor project, I wanted our team to build an interactive, fun musical application instead of a basic calculator. I researched how 8086 computers produce sound and discovered that **emu8086 can control the computer's PC speaker** directly by sending output signals to the sound ports. By dividing the clock frequency by musical notes, we can make the PC speaker play exact musical tones.

However, while testing in emu8086, we noticed an obvious problem: the PC speaker can only make **electronic beeps and buzzes**. It cannot play real instrument sounds like a genuine acoustic piano or guitar. So our team decided on a clear workflow: **we will create two connected parts for our project**. First, we wrote the full assembly language code in emu8086 to prove low-level hardware control. Second, we built a modern Web Studio using **HTML, CSS, and JavaScript**, where we added **28 real studio recordings** of real instruments so our band sounds completely real.

In emu8086, I built the **Concert Piano module (piano.asm)**, drawing 3D white and black keys that press down when you type Q through I on your keyboard, with an octave switcher and a sustain pedal. In our HTML and JavaScript studio, I integrated **real recordings of a Steinway Grand Piano**, so when you click the keys, it plays authentic concert piano music.

I will now pass the presentation to Sanskar to explain the Guitar module."

TEACHER RAPID-FIRE DEFENSE Q&A; (30-SECOND ANSWER):

Teacher: "Akash, what was your role and how did you decide on this project?"

Your Answer: "Sir/Ma'am, I led the project research and workflow. I researched how emu8086 controls the PC speaker to play music. Because the PC speaker only gives basic beeps, I decided we should build the assembly code in emu8086 and also create a realistic Web Studio in HTML, CSS, and JavaScript with real recorded instruments. I programmed the Piano in assembly, added real Steinway piano sounds to the web studio, and assembled the whole team's work together."

SPEAKER 2: SANSKAR

ASSIGNED ROLE: ACOUSTIC GUITAR MODULE (EMU8086 ASSEMBLY & WEB STUDIO FRETBOARD)

WHAT TO SAY (SPEAKS EXACTLY ONCE):

"Thank you, Akash. I designed and coded the **Acoustic Guitar module (guitar.asm)** in emu8086, and helped create the guitar interface in HTML, CSS, and JavaScript.

In emu8086 assembly, I wanted the guitar to feel like a real instrument rather than flat beeps. I programmed all **6 guitar strings from Low E to High E**. To make it sound like an acoustic string, I added a **pick strike chirp effect** when you pluck any string using keys Q through Y, followed by a gentle **finger vibrato effect** where the sound wobbles naturally. I also programmed the **[S] key to strum a full acoustic chord** across all strings.

In our HTML, CSS, and JavaScript studio, I brought this guitar to life visually. We searched online and downloaded **real acoustic guitar recordings of a Martin D-28 guitar**: all 6 open strings plus strummed acoustic chords for E minor, G major, C major, and D major. When you click or press strings in the web studio, the strings physically vibrate on screen using CSS animations, and it plays authentic acoustic guitar sounds.

I will now hand over to Krishna to present the Drums module."

TEACHER RAPID-FIRE DEFENSE Q&A; (30-SECOND ANSWER):

Teacher: "Sanskar, what did you do for the guitar?"

Your Answer: "Sir/Ma'am, in emu8086 assembly, I coded the 6 guitar strings, added pick strike sound effects, singing vibrato, and full chord strumming on key [S]. In HTML, CSS, and JavaScript, I built the guitar fretboard with vibrating strings and connected it to real Martin acoustic guitar recordings so it sounds authentic."

SPEAKER 3: KRISHNA

ASSIGNED ROLE: DRUMS MODULE (EMU8086 NOISE GENERATOR & WEB STUDIO 3D DRUMS)

WHAT TO SAY (SPEAKS EXACTLY ONCE):

"Thank you, Sanskar. I developed the **Drum Kit module (drums.asm)** in emu8086, and built the 3D drum kit stage in HTML, CSS, and JavaScript.

The biggest challenge in emu8086 was that the PC speaker only plays musical tones—it cannot naturally make drum noise like snares or cymbals. To solve this in assembly, I created a **random white noise generator in 8086 code**. Using this noise generator, I created 5 distinct drum sounds in emu8086: a deep **Kick Drum** that drops rapidly in pitch for an acoustic thump, a **Snare Drum** that blends white noise with a wooden shell sound, crisp **Hi-Hats**, and a shimmering **Crash Cymbal**.

In our HTML, CSS, and JavaScript studio, I connected these drums to **real close-miked studio recordings of a Ludwig Drum Kit**. I designed the circular 3D drum pads so when you hit keys Q, W, E, R, or T, the drum pads light up with expanding shockwaves and play punchy, realistic studio drum beats.

I will now pass to Hari to explain our Pre-Installed Songs."

TEACHER RAPID-FIRE DEFENSE Q&A; (30-SECOND ANSWER):

Teacher: "Krishna, how did you make drum sounds on a PC speaker?"

Your Answer: "Sir/Ma'am, because a PC speaker can only make musical tones, I wrote a random noise generator in 8086 assembly to create white noise for the snare and cymbals, and used quick frequency drops for the kick drum thump. In the HTML/CSS/JS studio, I connected these pads to real Ludwig drum kit recordings with moving shockwave animations."

SPEAKER 4: HARI

ASSIGNED ROLE: SONGS REPERTOIRE (EMU8086 TIMING & WEB AUDIO NOTE TABLES)

WHAT TO SAY (SPEAKS EXACTLY ONCE):

"Thank you, Krishna. Ghansham and I worked together on the **Pre-Installed Songs module (songs.asm)** in emu8086, and connected our music into the JavaScript Web Studio.

In emu8086 assembly, I stored the musical notes and timing for **6 famous songs**: *Happy Birthday*, *Twinkle Twinkle Little Star*, *Ode to Joy*, *Jingle Bells*, *Mary Had a Little Lamb*, and *London Bridge*. Each note is stored as two numbers: the note frequency and how long it should play. I used timer delay loops so every song plays at a steady, natural musical tempo.

In our HTML and JavaScript studio, I mapped these exact same song notes into our JavaScript audio player. This allows our web studio to play all 6 songs using the real Steinway piano, Martin guitar, or Ludwig drum beat across multiple octaves with 100% accurate pitch.

I will now pass to Ghansham to explain the Jukebox player and visualizer."

TEACHER RAPID-FIRE DEFENSE Q&A; (30-SECOND ANSWER):

Teacher: "Hari, what was your role in the songs module?"

Your Answer: "Sir/Ma'am, in emu8086 assembly, I encoded the musical notes and delay timings for 6 classic songs, calibrating the timer loops so songs play at steady tempo. In JavaScript, I mapped these same notes so they trigger real piano, guitar, or drum sounds in the web studio."

SPEAKER 5: GHANSHAM

ASSIGNED ROLE: JUKEBOX PLAYER, VISUALIZER & CUSTOM MUSIC COMPOSER (EMU8086 & WEB)

WHAT TO SAY (SPEAKS EXACTLY ONCE):

"Thank you, Hari. Building on Hari's song notes, I created the **Jukebox Song Player, Moving Visualizers, and the Custom Music Composer** in both emu8086 and HTML/CSS/JavaScript.

In emu8086 assembly (demo.asm), I wrote the automatic player that reads through the song notes, sends sound to the PC speaker, and animates **8 colorful equalizer bars** bouncing up and down on screen. I also made sure that pressing the **[ESC] key stops the song immediately** and returns cleanly to the main menu without freezing the emulator.

User Music Creation (Professor's Request):

Per our faculty guide's suggestion, we added a feature where users can create their own music just like the pre-installed songs! I developed the **Custom Music Composer module (composer.asm)** in emu8086. Users can play keys A through K, and the notes are recorded directly into 8086 RAM memory as frequency and duration pairs. You can change note lengths, undo notes, and play your custom created song immediately! Your custom song also appears as **Option 7 in the Jukebox Repertoire**, so you can play your own song in Grand Piano, Lead Guitar, or Drum style with dancing equalizer bars!

In our HTML and JavaScript studio, I upgraded this into a **16-Step Multi-Track Sequencer and 60 FPS Spectrum Equalizer**. Users can click grid squares across 8 instrument tracks to compose custom looping beats and melodies with real studio sounds.

I will now pass back to Akash to present our system assembling and live demonstration."

TEACHER RAPID-FIRE DEFENSE Q&A; (30-SECOND ANSWER):

Teacher: "Ghansham, how can users create their own music in your project?"

Your Answer: "Sir/Ma'am, in emu8086 assembly, I developed composer.asm where notes played on the keyboard are stored into a RAM buffer as frequency and duration pairs. Users can edit note lengths and play their custom song in Piano, Guitar, or Drum style with the visualizer! In the Web Studio, I built a 16-step sequencer matrix across 8 instrument tracks that loops your custom song with real acoustic samples."

SPEAKER 6: AKASH (CLOSING & DEMO)

ASSIGNED ROLE: PROJECT ASSEMBLING & LIVE DEMONSTRATION

WHAT TO SAY (SPEAKS EXACTLY ONCE):

"Thank you, Ghansham. To conclude our presentation, I will explain how we assembled the complete project and how to run it.

How We Assembled Everything:

In emu8086, I linked all our team files together inside kernel.asm using include statements: bringing together piano.asm, guitar.asm, drums.asm, and songs.asm into one cohesive program. You can run it directly in emu8086 by opening kernel.asm, clicking **Emulate**, and clicking **Run**.

To make our Web Studio equally easy to run, I compiled the HTML, CSS, JavaScript, and all **28 real instrument recordings into a single standalone file: CrimsonOrbit_RealStudio.html**. It sits right in our Downloads and Desktop folder. You just double-click it, and it immediately opens in Chrome or Edge, playing real Steinway piano, Martin guitar, and Ludwig drums completely offline.

We will now demonstrate both: first our assembly code running in emu8086, and then our Real Sound Studio. Thank you, Professor, and we are ready for your questions!"

TEACHER RAPID-FIRE DEFENSE Q&A; (30-SECOND ANSWER):

Teacher: "Why does your project have both emu8086 assembly and an HTML/CSS/JS file?"

Your Answer: "Professor, our core project is 100% written in 8086 assembly language running in emu8086. But because the PC speaker in emu8086 only makes electronic beeps, we also built the HTML, CSS, and JavaScript studio to show how our exact assembly music sounds with real studio-recorded instruments like Steinway piano and Martin guitar. This proves we mastered both low-level assembly and modern audio presentation!"
